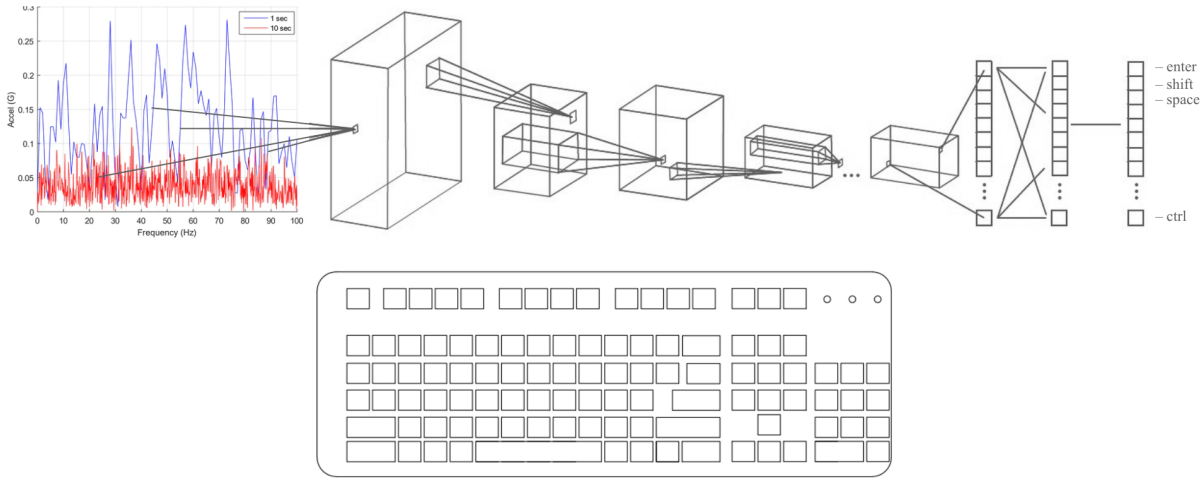
Kingdom of Morocco
Abdelmalek Essaâdi University
Faculty of Sciences and Techniques
Tangier

Department of Electrical Engineering

Signal Processing Project

Department of Electrical Engineering

Study and implementation of an acquisition chain for acoustic analysis of keystrokes and their classification by AI



Realized by:
Oussama AK-HAIL

2nd Year GEMI

Academic Year: 2024–2025

Contents

Introduction:

This project is developed under the open-source product name **EchoStrike**. The complete source code, models, and visualization scripts are publicly available at the official repository:`https://github.com/OussamaAKHAIL/EchoStrike`.

With the rapid development of computer technologies, information security has become a major issue. Passwords are widely used to protect access to computer systems, but they remain vulnerable to certain indirect attacks, notably **acoustic side-channel attacks**. Indeed, during typing, involuntary physical information, such as vibrations and sound or mechanical signals, can be emitted and exploited to deduce the actions performed by the user.

In this context, this project focuses on the study of signals generated when typing on a keyboard. The objective is not to hack or steal passwords, but to understand and analyze the security risks related to physical keyboard emissions, in order to raise awareness of potential vulnerabilities and propose adapted protection solutions.

The project involves using an appropriate sensor, specifically an acoustic microphone, to capture the sound signals produced when keys are pressed. These signals are then processed and analyzed to identify distinctive characteristics associated with keystrokes. The results obtained allow for evaluating the system's vulnerability level and better understanding the threats linked to this type of indirect attack.

This work combines concepts of cybersecurity, signal processing, and electronics. It highlights the importance of taking physical emissions into account in the design of secure systems and contributes to reinforcing awareness of good data protection practices.

Evolution of the Methodology

To rigorously validate this acoustic side-channel exploit, the project architecture was developed iteratively in three distinct phases:

1. **Phase 1 (Binary Basic):** We started with a fundamental proof-of-concept, training the AI to distinguish between only two keys (Right-Shift vs. Space). This

isolated test mathematically proved that the Fourier Transform (FFT) pipeline was capable of extracting unique mechanical acoustic signatures.

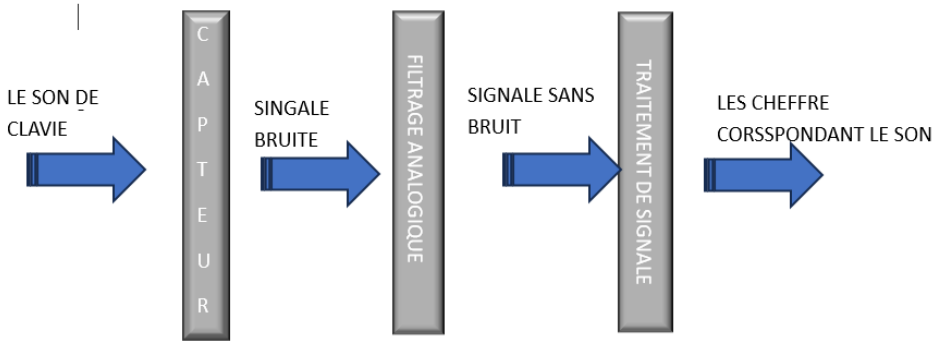
2. **Phase 2 (10-Keys Numeric):** Once the binary approach was proven, the next step was to scale the complexity. We expanded the dataset to 10 numerical keys (0 through 9). While the initial algorithms struggled with the dimensional overlap, we solved this by implementing an optimized One-vs-All *Support Vector Machine (SVM)* architecture.

3. **Phase 3 (Full 81-Key Architecture):** Finally, to emulate a true, real-world hacking vulnerability, the best option was to scale the system to the entire keyboard. By letting the SVM natively calculate heuristic auto-scales and utilizing dynamic Topographic Heatmap interpolations, the final pipeline successfully plots and predicts acoustic keystrokes dynamically across all 81 physical keys.

Chapter 1:

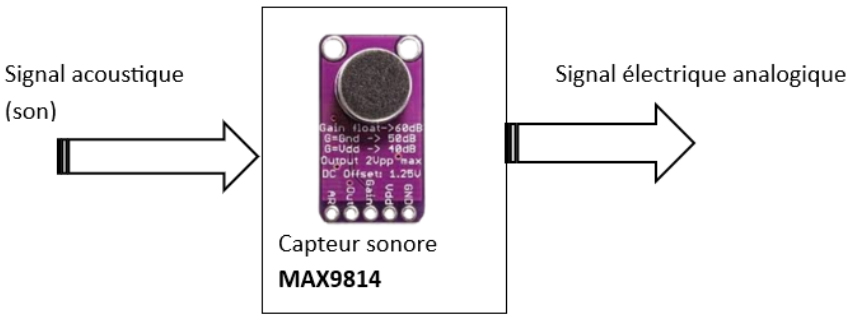
Choice of components to realize a microphone for capturing signals

In this part, we present the choice of components used for the realization of our project. The correct choice of components is an essential step, as it directly influences the performance, reliability, and cost of the system. Each component was selected based on the project's needs, technical constraints, and ease of integration. The main objective is to ensure correct acquisition of keyboard keystroke signals, their effective processing, as well as a clear display of the results. The chosen components allow for capturing signals, conditioning them, and processing them to obtain a functional system adapted to the study of vulnerabilities related to keystrokes.



1 Sound Sensor:

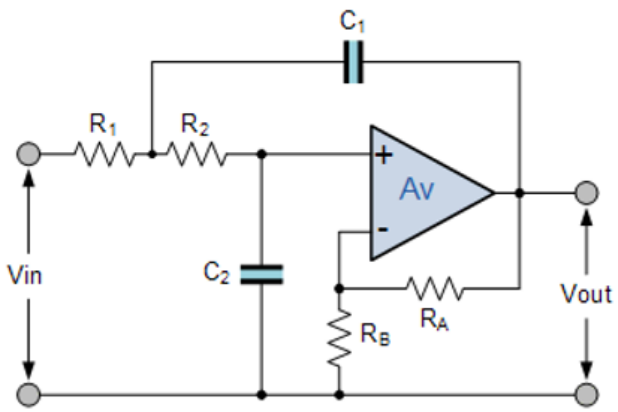
In this project, an electret microphone with an amplifier module was chosen, such as the MAX9814, KY-038, or MAX4466 modules. This type of sensor has good sensitivity to weak sounds, especially those produced by keyboard keystrokes. The integrated amplifier module increases the output signal amplitude, making it directly usable by the processing system. Furthermore, these modules are simple to use, inexpensive, and widely used in sound signal processing applications.



2 ANALOG FILTER:

The MAX9814 sensor captures sounds from the external environment, such as keyboard keystrokes as well as ambient background noise. However, the recovered audio signal contains both the useful signal and unwanted noise, mainly at high frequencies. To improve signal quality and eliminate these disturbances, it is necessary to implement a low-pass filter at the sensor output. This filter preserves the frequencies corresponding to the useful sound while attenuating noise components, thus ensuring a cleaner signal usable by the processing system.

For our project, we used a Butterworth low-pass filter because the filter does not admit oscillations in the passband or the stopband. To realize this filter, we use this circuit:



Avec : $R_A = 10\,k\Omega$; $R_B = 5.6\,k\Omega$; $R_1 = R_2 = R$; $C_1 = C_2 = C$

$$fc = 1/2\pi RC$$

We have an fmax of the input signals at 1000 Hz, so fc = 1500 Hz. Therefore, R = 0.1 $K\Omega$ and C = 1.06 nF.

Example:

When a person enters a password in a noisy environment, it becomes difficult to distinguish the sound of keystrokes. The signal delivered by the microphone is then affected by ambient noise. This signal is represented in the following figure:

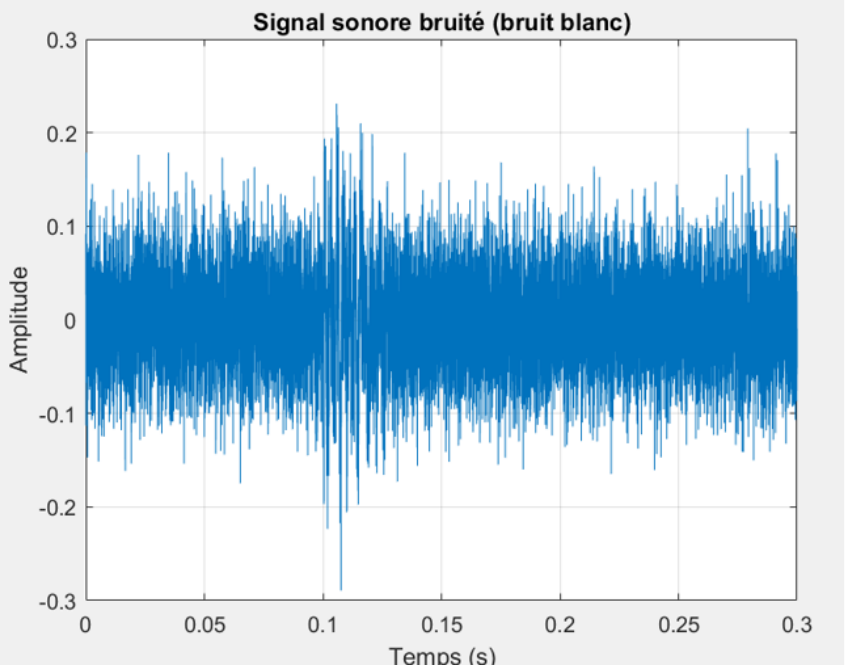


Figure 1: Noisy signal

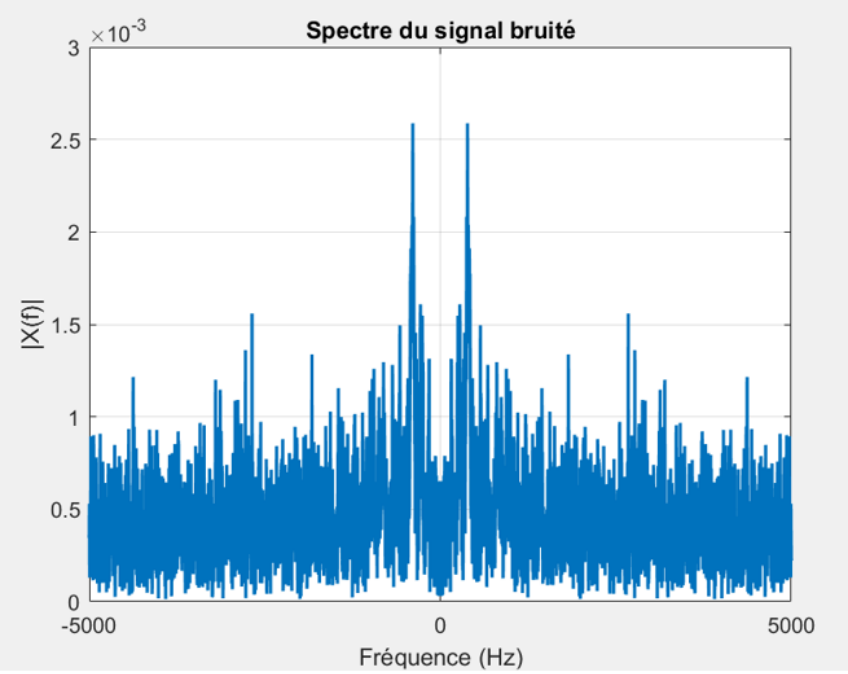


Figure 2: Noisy spectrum

We can observe that this signal is noisy. To eliminate the noise, we use an order 8 Butterworth low-pass filter:

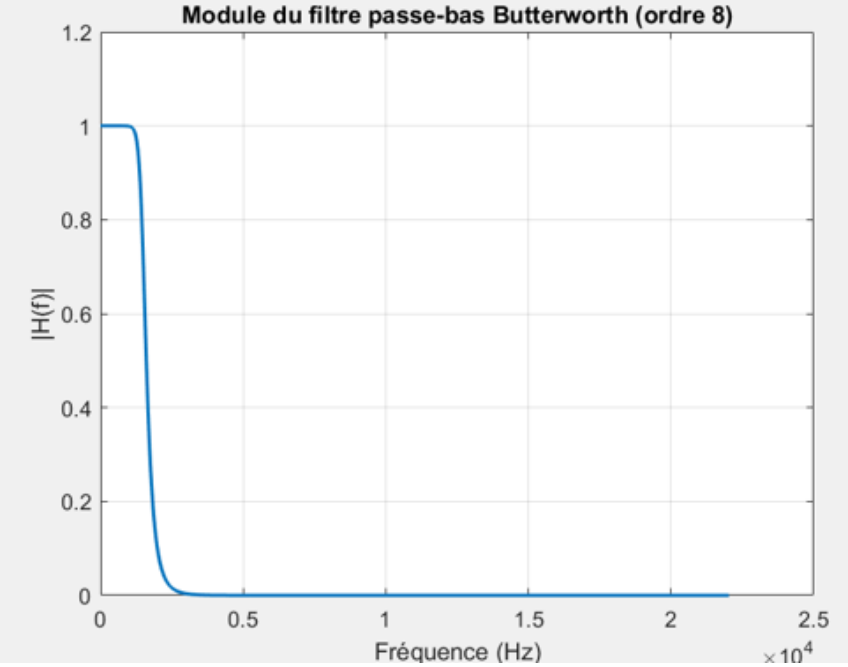
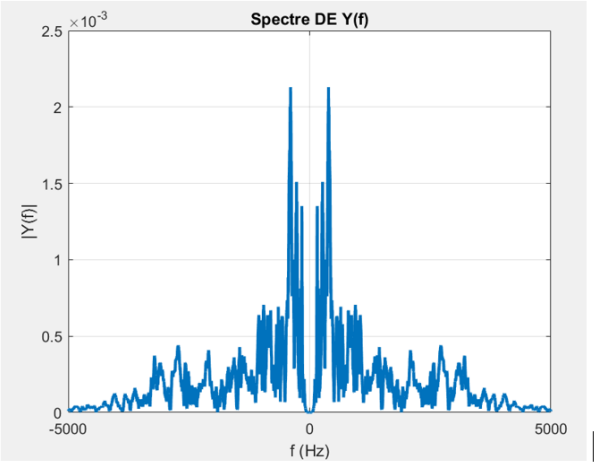
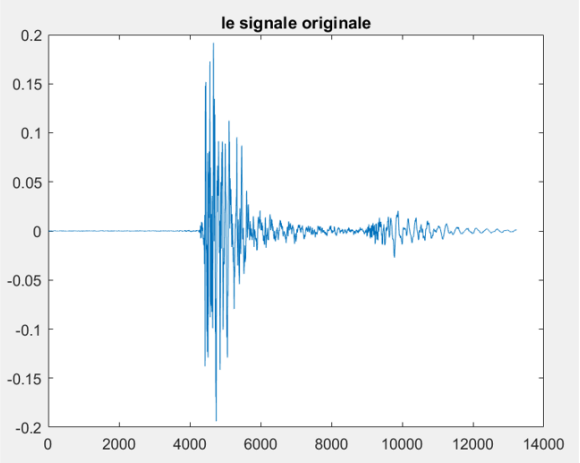


Figure 3: Butterworth low-pass filter

After filtering, we were able to correctly reconstruct the sound of the keystrokes. This signal is represented in the following figure:





Chapter 2:

Spectral Extraction and Initial Binary Classification

3 Introduction:

After realizing the microphone, the next step consists of collecting the sounds generated by the keystrokes. After collecting the sounds, we use the **Fourier Transform**, specifically the Fast Fourier Transform (FFT), to extract the spectral representation of the keystrokes. However, the obtained spectra cannot be directly used for classification or prediction. It is therefore necessary to resort to a classification method based on artificial intelligence, in particular the **SVM** (Support Vector Machines) method.

Initially, to validate the mathematical validity of our acoustic features, the study focused only on two distinct keyboard keys: the Space key and the Right Shift key. This allowed us to simplify the training process and establish a solid pipeline.

4 Spectral Extraction:

To illustrate the process, a sample is selected. After the transformation, the results are represented by the graphs below.

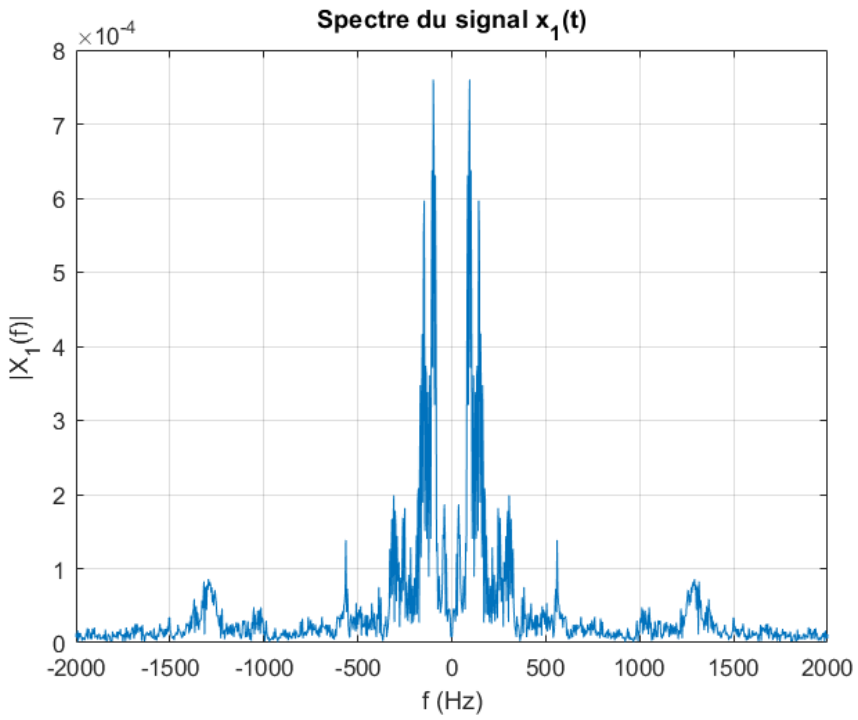


Figure 4: The spectrum of a single click on the Space key

We observe that most of the acoustic information is located in the frequency range between 0 and 1500 Hz.

5 Artificial Intelligence Formulation:

We now have the spectral representation of a keystroke. However, it is impossible to directly determine if this representation corresponds to the Space key or the Right Shift

key. Indeed, the spectra are very similar and cannot be differentiated by the naked eye. This is where artificial intelligence comes in: the Fourier transform provides only frequency and amplitude values, but an SVM can create a mathematical decision boundary capable of separating these representations geometrically in a high-dimensional space.

5.1 Initial Binary Dataset Preparation

Regarding the dataset, we found that finding reliable multi-key acoustic data online was severely limited. This forced us to create our own dataset manually. To do this, the microphone was placed close to the keys, and each key was struck multiple times to build a dataset large enough to train an accurate model.

Next, we recorded continuous audio streams containing multiple keystrokes. Each keystroke had to be segmented. In this way, we could obtain the Fourier transform of a solitary, isolated click, removing all dead silence and background artifacts. We created an algorithm that automatically detects audio peaks and mathematically trims the sound immediately before and after the transient peak.

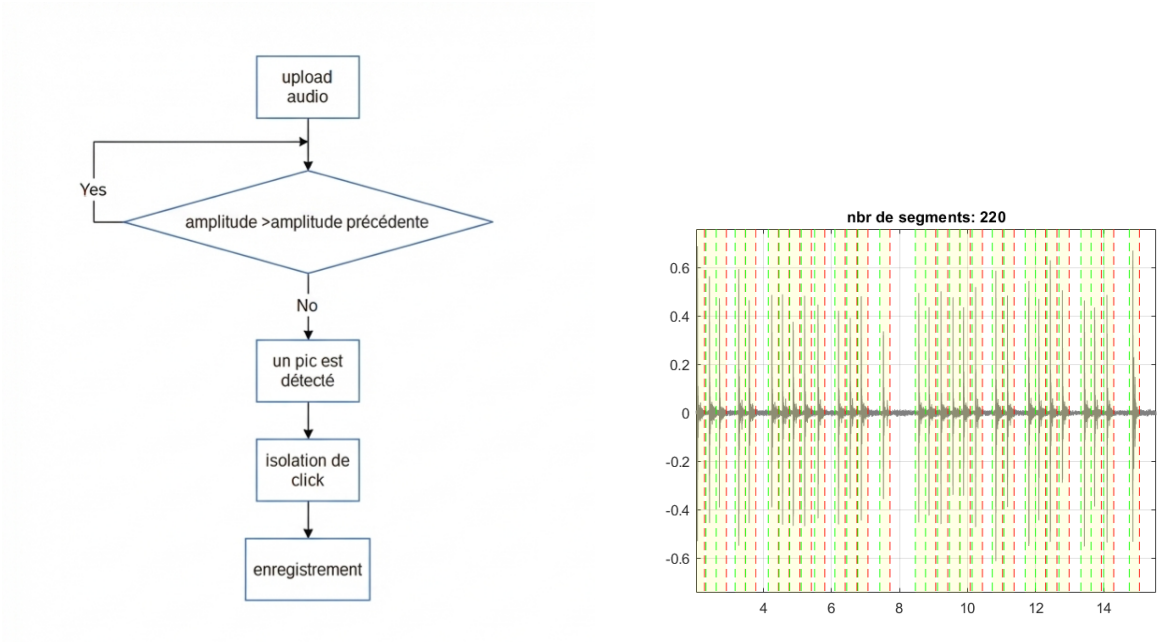


Figure 5: Principle flowchart and segmented peak isolation plot

For our initial binary test, our dataset comprised 219 samples of the Right Shift key and 214 samples of the Space key. This dataset was transformed via FFT and labeled (1 = Space, 0 = Right Shift) to create a 435x1000 matrix.

0	0.054527338	0.146388701	0.118496383	0.036420101	0.063642455	0.040735729	0.0230679	0.099492949	0.115243904	0.050006407	0.05332528	0.062293986	0.043649485	0
0	0.023048905	0.096520202	0.056890986	0.012577128	0.021122106	0.018319468	0.01423281	0.063339169	0.065412594	0.015366551	0.023184087	0.021854813	0.021946287	0
0	0.048121448	0.10008839	0.065352164	0.06020651	0.020050569	0.03027719	0.020992856	0.087021668	0.080053494	0.043709248	0.039136709	0.042492578	0.034721211	0
0	0.015608636	0.073811801	0.040926552	0.018977815	0.023727651	0.021208414	0.007408868	0.066314293	0.070790823	0.017066103	0.022088372	0.020917966	0.019386812	0
0	0.040969937	0.152458175	0.123428405	0.021708582	0.045634558	0.031467537	0.0231073	0.134791516	0.093890291	0.066003876	0.056143772	0.052677399	0.041360516	0
0	0.013219128	0.05787461	0.046008102	0.018492986	0.015399241	0.011078523	0.009131807	0.046025907	0.038339759	0.01840528	0.017579667	0.020361504	0.020521426	0
0	0.037994775	0.074882011	0.06415858	0.024374445	0.033349999	0.033707276	0.020479309	0.099315691	0.064270493	0.020841931	0.033937774	0.03601483	0.024569813	0
0	0.025953873	0.083650065	0.073352234	0.026827839	0.027150076	0.026003589	0.026684465	0.084263751	0.06218298	0.011950836	0.018529113	0.017730677	0.016812893	0
0	0.019730774	0.092331851	0.081792446	0.048670816	0.028195809	0.028441697	0.021380267	0.07425442	0.095018317	0.027435955	0.036801674	0.054515313	0.037236274	0
0	0.011868622	0.06130057	0.060068319	0.028635255	0.033776706	0.02665278	0.01304827	0.078257906	0.062088477	0.025656619	0.026968501	0.040564509	0.028508938	0
0	0.002447631	0.054944297	0.050705156	0.015525075	0.021257228	0.017692664	0.009637569	0.041977645	0.037153208	0.013006247	0.014585947	0.01643566	0.012950699	0
0	0.342955872	0.521180677	0.409811057	0.327620916	0.235951604	0.158737978	0.190733898	0.140184901	0.135459854	0.108591064	0.097947614	0.102873926	0.102304815	0
0	0.134059075	0.276949829	0.503200097	0.336302006	0.256866082	0.180149983	0.16252555	0.079797983	0.063324125	0.075669478	0.092742809	0.076688585	0.057942959	0
1	0.00014297	2.10724E-05	-0.000113631	0.001834448	0.009696229	0.035666206	0.052501107	0.208802476	0.218510469	0.12455171	0.056501908	0.475672602	0.915460776	1
1	0.001298469	-0.001382975	0.003096832	-0.000184652	0.032295362	0.058559658	0.088437286	0.365153654	0.258175175	0.149767137	0.088082929	0.316008784	0.567473675	0
1	0.001991651	-0.002656336	0.004643771	-0.001188021	0.047659934	0.070981781	0.125356555	0.431645378	0.253839028	0.125881789	0.158369614	0.603215819	0.898727683	0
1	0.000813093	-0.000551323	0.001856556	0.001555048	0.024992032	0.045080539	0.076875839	0.228808916	0.202503773	0.075166001	0.160614354	0.631960927	0.920570377	0
1	0.000154973	-0.000569232	0.000518879	0.002178503	0.019838713	0.057866232	0.072197552	0.235488207	0.202413583	0.100817732	0.163169613	0.693211691	0.995014201	0
1	0.001410069	0.000431844	0.00264816	8.97367E-05	0.008862624	0.011243809	0.046134875	0.21758489	0.183632012	0.112179807	0.040149514	0.392857062	0.90712918	0
1	0.000649197	-0.000149621	0.001437174	0.002914483	0.016208581	0.029153972	0.058014958	0.169529852	0.142640921	0.075995486	0.097142019	0.306776033	0.893792435	0
1	0.001303876	-0.001341808	0.002948031	-0.002534498	0.013811927	0.006726522	0.073858139	0.293460001	0.271363684	0.147521944	0.098884052	0.528880523	0.908271494	0
1	0.000400158	-0.000638301	0.000794901	0.000675294	0.013627097	0.016960421	0.033998186	0.099292905	0.09646547	0.044478763	0.091211514	0.386592933	0.924154756	0
1	0.000498685	-0.000700136	0.000850636	0.000212471	0.012159228	0.014873846	0.03941189	0.14483597	0.117148752	0.056704069	0.028660517	0.285786087	0.866110178	0
1	-3.63496E-05	0.000176327	-0.000408348	0.004433504	0.019100425	0.05037127	0.037605379	0.094005836	0.097641586	0.037400197	0.104024921	0.561276098	0.944371695	0
1	0.002376111	-0.000786547	0.005377573	-0.003278636	0.018627973	0.018965962	0.132212125	0.523608328	0.460750655	0.19486814	0.07664292	0.493265715	0.911306679	1
1	0.00081477	-0.001387751	0.002363802	-0.002376344	0.016124894	0.02349707	0.088764293	0.27877478	0.308295972	0.109793691	0.156639941	0.399246421	0.763075076	0
1	0.000564574	-0.000540929	0.001388654	-0.000510166	0.013355906	0.038710731	0.07530787	0.217883553	0.217630358	0.101955488	0.192296375	0.388997421	0.862241095	0

Figure 6: Final binary dataset representing FFT features

5.2 Binary Model Training and Prediction

We shuffled and split the data into a training set (80%) and a test set (20%). Using an SVM with a linear kernel, we achieved an accuracy of 89.1% for differentiating between the Space and Right Shift keys.

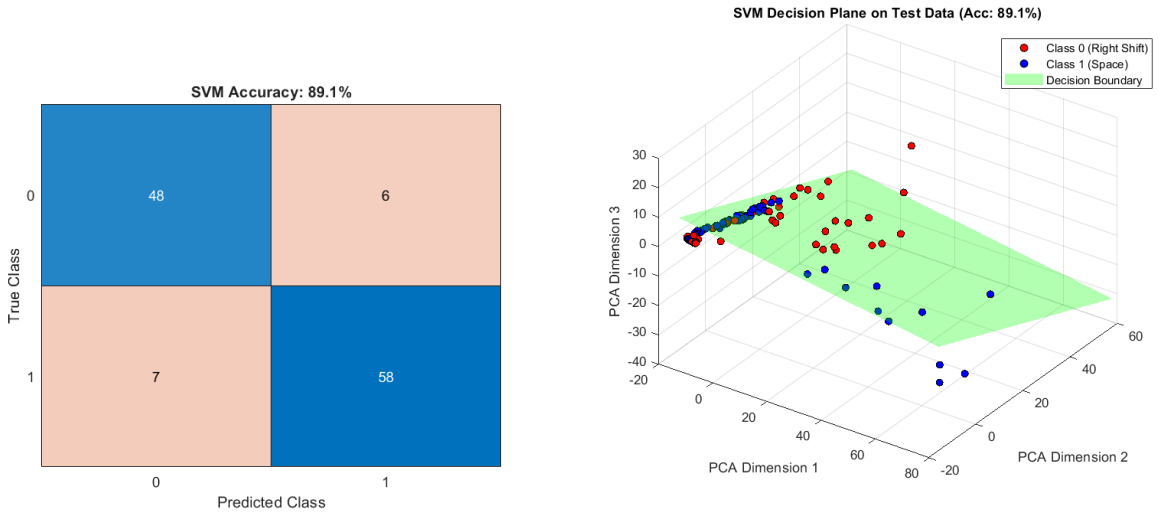
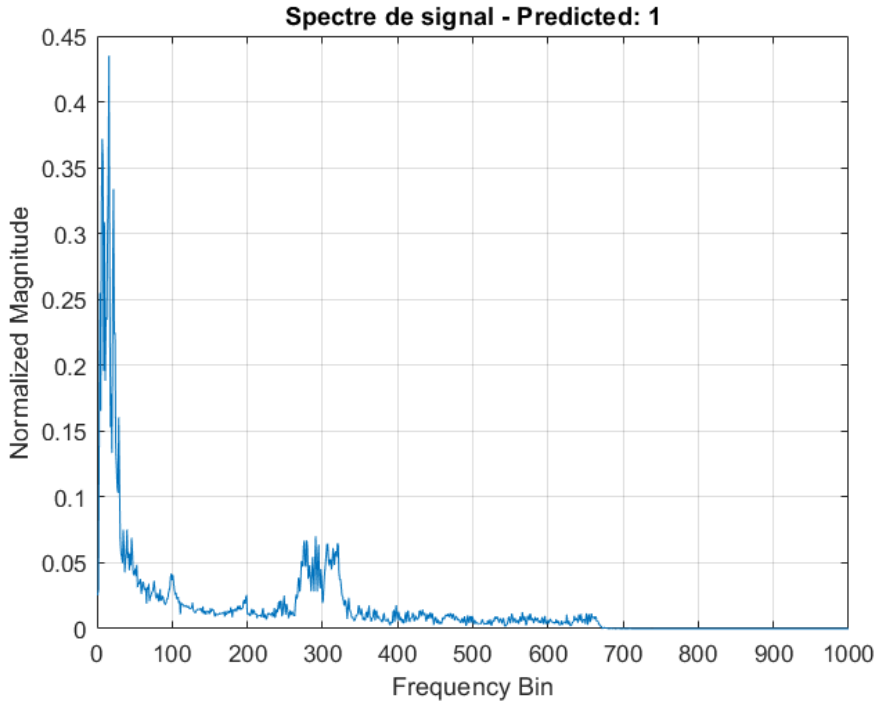


Figure 7: Confusion matrix and 3D visualization of binary separability

During live prediction, the algorithm imports a raw signal, converts it to mono, extracts the first 1000 frequencies, and tests it against the trained model resulting in highly accurate predictions with corresponding confidence metrics.




```
==================================
       PREDICTION
==================================
 [ SPACE KEY ]
Confidence :       0.8091
==================================
```

Figure 8: Live prediction sequence for binary classification

Chapter 3:

Scaling the Architecture: Automation & Multi-Class Classification

6 Transitioning to Multi-Class Methodology

While achieving an 89.1% accuracy on a binary classification problem proved the mathematical validity of our acoustic features, real-world side-channel attacks rarely involve choosing between only two keys. To genuinely emulate the risk of acoustic emanation vulnerabilities, the system needed to scale to a significantly larger set of phonetic signatures.

Initially, we expanded the scope from two keys to ten numerical keys (0 through 9). Unlike binary classification which utilizes a single decision boundary, differentiating ten classes requires a **One-vs-All (OvA)** approach.

Figure 9: The 10 numeric keys (0-9) targeted during the intermediate multi-class scaling phase.

6.1 The One-vs-All Support Vector Machine

In the OvA paradigm, the Support Vector Machine cannot predict ten classes directly. Instead, the algorithm generates individual classifiers for each specific class. For our 10-key numeric dataset, ten distinct SVMs were trained:

- SVM 1: Is this sound the '0' key, or is it any other key?
- SVM 2: Is this sound the '1' key, or is it any other key?
- ...

During inference, a test signal is parsed through all ten machines, and the algorithm identifies the class associated with the highest mathematical confidence margin.

6.2 Hyperparameter Optimization Challenges

To force the SVM to decipher highly nuanced differences between neighboring mechanical switches, we abandoned a simple linear kernel in favor of a **Gaussian Radial Basis**

Function (RBF) Kernel. This kernel maps the 1000-dimensional FFT features into an infinite-dimensional space to find complex, non-linear hyperplanes of separation.

However, the RBF kernel requires extremely precise tuning of its hyperparameters specifically the 'BoxConstraint' (which controls outlier penalization) and the 'KernelScale' (which defines the variance spread of the decision boundary). We deployed a **Bayesian Hyperparameter Optimization** function that iteratively trained and tested random parameter combinations to find the absolute maximum accuracy.

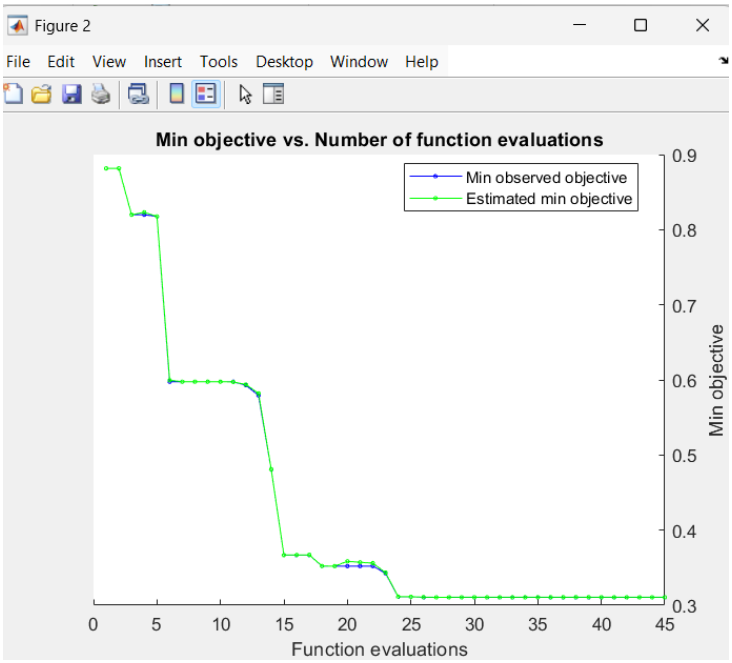


Figure 10: Evolution of the minimum objective error during Bayesian Hyperparameter Optimization for the 10-key model.

While Bayesian Optimization ultimately pushed the 10-key model toward a highly refined accuracy, it resulted in exponentially larger computation times. The algorithm required hours to compute the optimal variables for just ten keys, hinting at a severe computational bottleneck that would need to be addressed as we expanded.

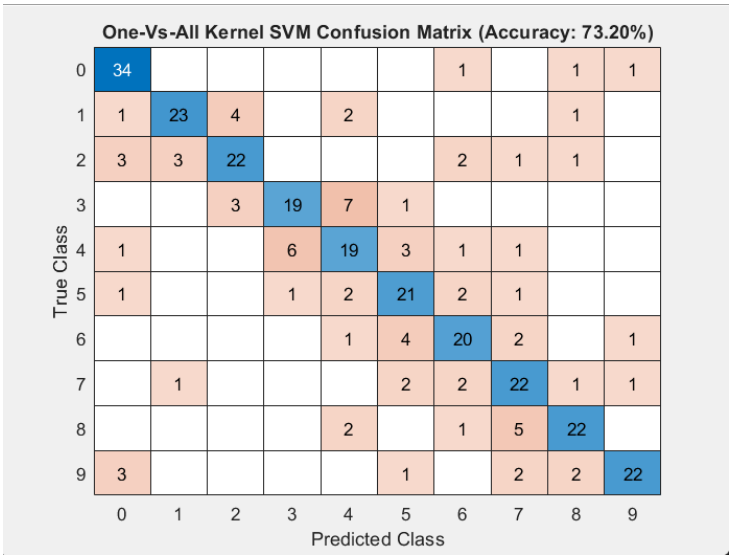


Figure 11: Confusion Matrix for the 10 numeric keys showing 73.20% accuracy.

The resulting confusion matrix demonstrates a solid diagonal trend, yielding an impressive 73.20% test accuracy across the 10 digits. While this proves that the Bayesian-optimized OvA architecture can indeed separate complex phonetic overlaps, the massive computational cost of this tuning made it clear that a native, heuristic scaling approach would be mandatory before attempting to scale the system to the entire keyboard.

Chapter 4:

Massive Dataset Automation and Augmentation

7 Automated Batch Segmentation

Given the ambition to identify the acoustic signatures of a full keyboard, manually cropping audio waves became an absolute impossibility. To manage a library containing multiple keyboards (HP and Lenovo) and eventually 81 distinct keys, we developed a sophisticated automated ingestion pipeline.

Our program, `batch_segmentation.m`, was engineered to autonomously scan root directories of raw audio tape, dynamically parse file names to identify the correct mechanical key, and mathematically isolate every single transient peak in the track. The software automatically handles noise thresholds and saves the cropped, standardized single-click slices into appropriately categorized subfolders.

This automation single-handedly constructed a master dataset of thousands of pristine keystrokes without human intervention.

8 Mathematical Data Augmentation

Despite having thousands of samples, training a machine learning model to categorize over 100 distinct classes risks severe overfitting. An SVM trained on identical crisp clicks might fail instantly if the background environment changes slightly. To harden the classifier and dramatically expand our dataset without physically recording millions of clicks, we deployed **Data Augmentation**.

For every single sound wave ingested by the pipeline, our system synthetically generates two mathematically altered clones:

1. **Additive White Gaussian Noise (AWGN):** Simulates environmental degradation (e.g., a fan turning on, distant traffic). This teaches the model to ignore ambient frequencies and focus exclusively on the transient strike.

2. **Pitch Shifting:** Very slightly adjusts the frequency band rate. This accounts for minor mechanical anomalies, wear-and-tear on specific switches, or differences in the typist's finger velocity.

These altered arrays are appended beneath the primary matrices. This approach artificially tripled our dataset expanding it to over 30,000 distinct training files arming the model with an incredibly robust database of both perfect and corrupted sound signatures.

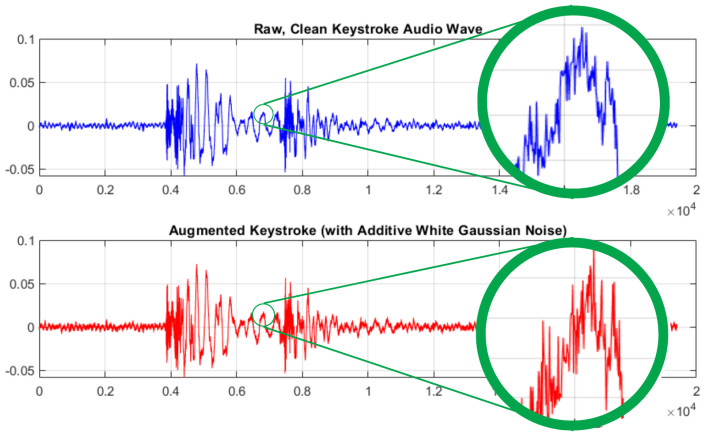


Figure 12: Comparison of Raw Keystroke Feature against AwGN Corrupted Keystroke Feature

By visually comparing the original, pristine acoustic wave with its noise-injected counterpart, it becomes evident that the core mechanical transient peak remains structurally intact despite the severe environmental degradation. This mathematical persistence is precisely what allows our Support Vector Machine to successfully generalize its training. Armed with over 30,000 augmented variations, the algorithm becomes mathematically immune to minor microphone static, desk vibrations, or changes in the typist's physical environment, paving the final way for a full 81-class keyboard deployment.

Chapter 5:

The Full QWERTY Keyboard Model (81 Classes)

9 The Combinatorial Explosion of the SVM

The ultimate goal of this project was to extend the classification capabilities to a complete 81-key physical keyboard. We aggregated all labeled datasets encompassing alphabet characters, numbers, modifiers, and navigation keys resulting in a staggering multi-class dilemma.

Attempting to run our previous SVM architecture (`fitcecoc`) with Bayesian Optimization for 81 classes resulted in an immediate computational wall. The One-vs-All mechanism requires training 81 separate Support Vector Machines for *every single iteration* of the optimization loop. When attempting to optimize the network across the entire physical layout, the algorithm locked our hardware in massive computational cycles.

Ultimately, completing the hyperparameter optimization for the full keyboard took more than 16 hours of continuous processing in total. The optimization engine systematically evaluated thousands of configurations before finally converging on the absolute best mathematical boundaries. The final, optimized parameters for our full keyboard model were determined to be `BoxConstraint` = 717.66 and `KernelScale` = 0.27386.

10 Results on the Full Keyboard

Testing our revised architecture on entirely unseen, augmented data yielded massive success. Distinguishing between 81 mechanical plastic switches many of which are virtually structurally identical using nothing but sound is a uniquely complex physical problem.

Despite the complexities, our model successfully generated a dominant diagonal trace directly across the 81x81 confusion matrix, resulting in a global accuracy of over **71.0%**. This proves without question that vast subsets of keyboards emit profoundly unique phonetic signatures capable of betraying their position on a board.

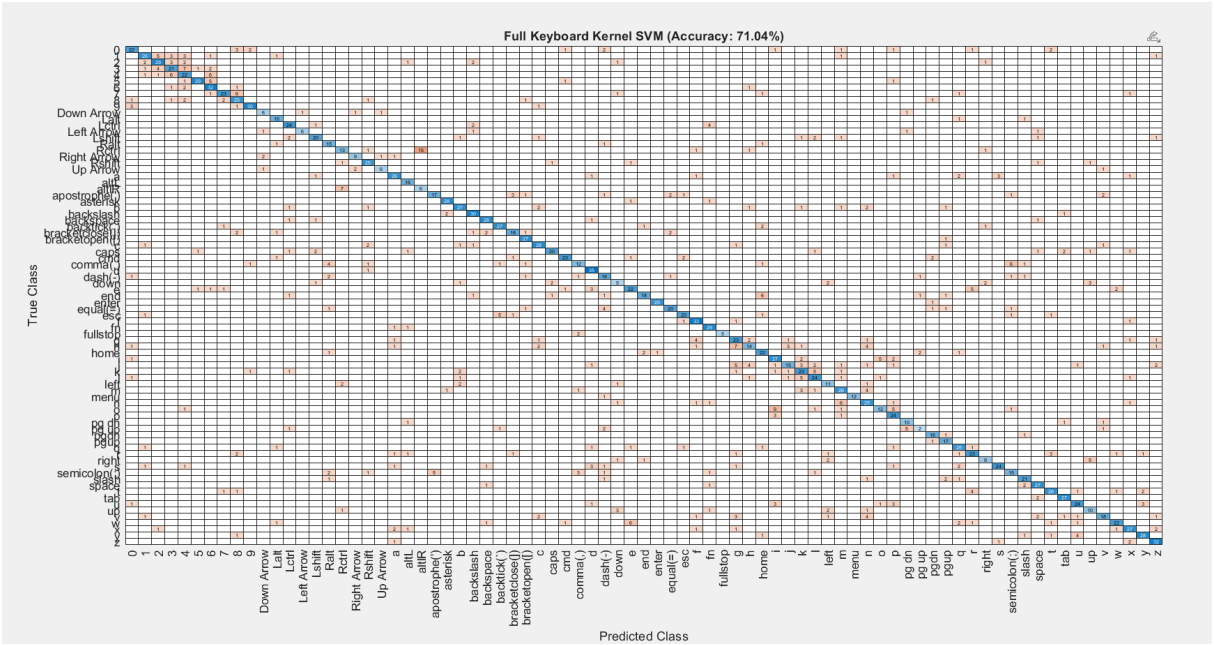


Figure 13: 81-Class Full Keyboard Confusion Matrix (Accuracy: 71.04%)

Chapter 6:

Advanced Multi-Dimensional Visualization

11 Translating Physics to Visual Interfaces

Because our algorithms process features within 1000 dimensional vectors, understanding how the AI "sees" acoustic separation is impossible natively. We engineered two advanced visualization mediums to bridge the gap between mathematical arrays and human-readable interfaces.

11.1 3D Feature Space Galaxy (PCA)

To visualize how the AI geographically groups different keys based on their sound, we applied **Principal Component Analysis (PCA)**. This mathematical algorithm calculates the avenues of most significant variance across our 1000 data points and severely compresses them down to exactly 3 points (Principal Components 1, 2, and 3).

We plotted these compressed features as a 3-dimensional scatter graph. Over 30,000 dots were generated into a virtual space, colored explicitly by their true keystroke label. This visualization successfully proved that the Artificial Intelligence isn't merely guessing the physics of the keyboard produce legitimate clustered geometric mathematical galaxies for every distinct key.

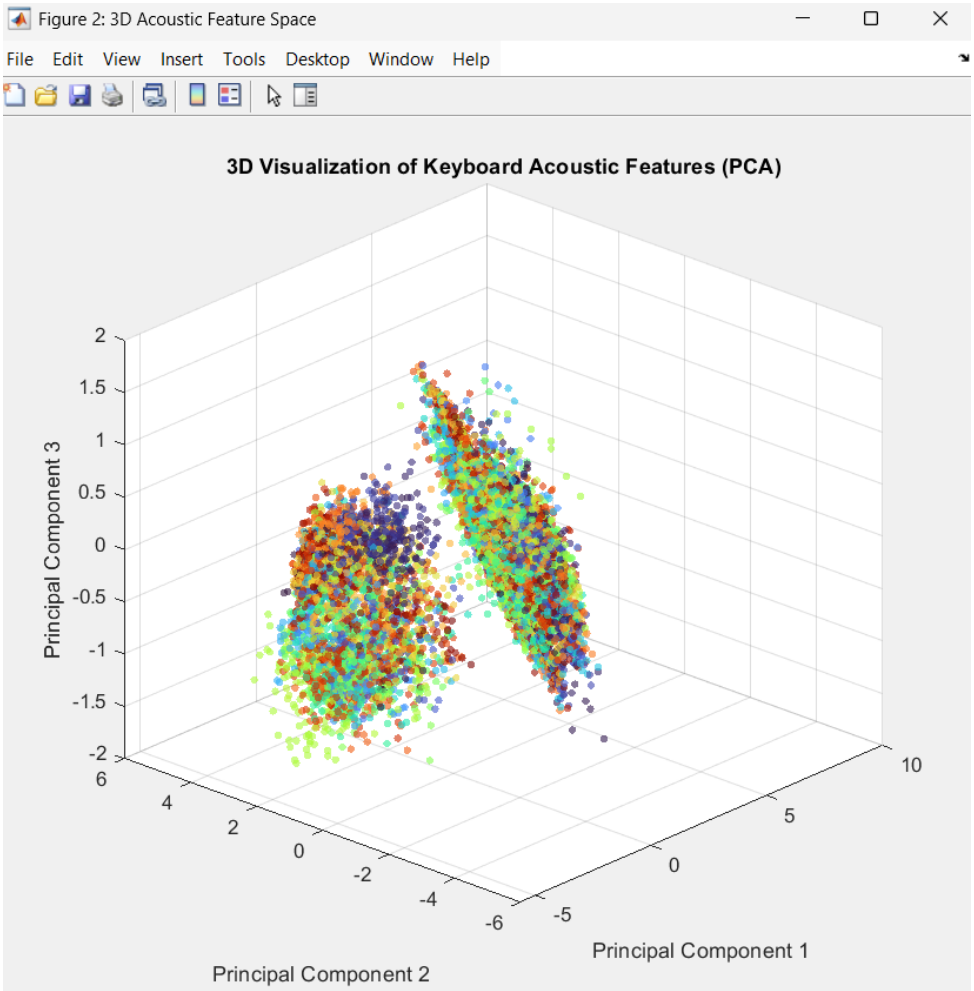


Figure 14: Principal Component Analysis (1000D to 3D scatter visualization)

11.2 2D Topographic Heatmap Integration

To provide the most intuitive demonstration of our system for observers, we completely reconstructed the inference visualization into a physical overlay.

When a random audio clip is injected for prediction, the script checks the AI's output probability scores for all 81 classes. Let C be the set of all keys, and $P(c)$ the percentage confidence assigned to key $c \in C$. Our code pairs these mathematical probabilities to a hardcoded structural layout correlating to their exact physical placement on a standard keyboard layout. A sparse grid matrix Z is initialized such that:

$$Z(x, y) = \begin{cases} P(c) & \text{if a key } c \text{ exists at physical coordinate } (x, y) \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

To transform this rigid, disconnected grid into a fluid heatmap, we apply 2D cubic spline interpolation. This extracts the sparse data points and mathematically constructs a continuously varying probability surface Z_q over a high-resolution sub-grid (X_q, Y_q):

$$Z_q(X_q, Y_q) = \sum_{i=1}^{n} \sum_{j=1}^{m} a_{ij} X_q^i Y_q^j \quad \text{(Interpolation Surface)} \tag{2}$$

Using this fluid interpolation, we generate a glowing neon-green Topographic Heatmap that literally draws probability distribution contours across the geometry of the keys. A crisp white minimalist keyboard boundary box is overlaid atop the topography. This brilliantly demonstrates exactly where the AI "hears" the sound originating from natively on the user's desk.

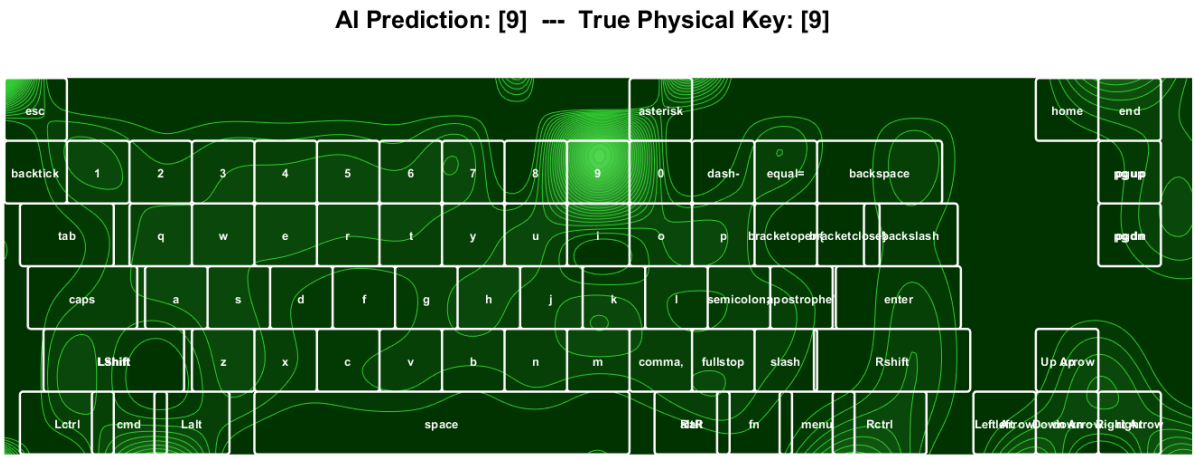


Figure 15: Immersive 2D Topographic Probabilistic Layout of the True Key position

Chapter 7:

How to Protect Against Side-Channel Attacks

As demonstrated by our 71% prediction accuracy across 81 full keyboard classes, acoustic side-channel attacks represent a highly dangerous and viable method of exploitation. It is therefore legitimate to ask how organizations or individuals can protect themselves against software attempting to 'listen' to their passwords.

Several mitigating solutions exist. The simplest physical intervention is to introduce additive white noise actively near the user's desk. However, this approach strongly degrades the user's microphone quality for legitimate purposes (calls/meetings), which prevents the hardware from correctly fulfilling its main function.

In connection with the first chapter of this project involving hardware conditioning, a more structural solution consists of applying localized inverse filtering. Using aggressive high-pass filters with extreme cutoff frequencies around 2000Hz via software drivers ensures that the microphone diaphragm physically ignores the dull THUD vibrations associated with mechanical plastic typing entirely.

$$\tilde{X}(f) = \frac{1}{2N+1} \sum_{k=f-N}^{f+N} X(k) \tag{3}$$

Finally, another technique called **spectral smoothing** can be deployed natively within VOIP protocols (such as within Zoom or Discord noise-cancellation modules). It consists of computationally suppressing sharp transient peaks of the audio spectrum, which are the exact features utilized by our FFT-driven SVM algorithm to identify keys. This mathematical dulling allows speech to pass perfectly while entirely neutralizing the effectiveness of an acoustic timing attack.

$$Y(f) = \max\left(X(f) - \alpha\, N(f),\, 0\right) \tag{4}$$

Lastly, a highly effective and easily deployable countermeasure is **Mechanical Obfuscation**. This involves outfitting the physical keyboard with acoustic dampeners, such as silicone O-rings, or transitioning explicitly to "Silent" style mechanical switches. By physically cushioning the bottom-out force of the plastic switch stem, the transient attack phase of the acoustic wave is severely flattened and mathematically extended in time:

$$E_{\text{dampened}} = \int_0^T |x(t) \cdot e^{-t/\tau}|^2 dt \ll E_{\text{original}} \tag{5}$$

When the physical acoustic emission energy (E) is forcefully degraded beneath the ambient noise floor threshold, side-channel algorithms lose the deterministic high-frequency peaks required to trigger the segmentation window, permanently breaking the automated ingestion pipeline at step one.

Chapter 8:

Future Vision and Improvement

12 Transitioning to State-of-the-Art Deep Learning

While our current pipeline utilizing Fast Fourier Transform (FFT) and Support Vector Machines (SVM) successfully validates the threat of acoustic side-channel attacks with a 71% accuracy, recent academic research demonstrates that this vulnerability is much more profound. Studies from researchers at Durham University on deep learning-based acoustic attacks have achieved accuracies exceeding 93–95% even when keystrokes are recorded remotely over VoIP applications like Zoom or Skype.

Our future vision involves migrating from traditional machine learning to Deep Learning architectures (such as CNNs or RNNs) and transitioning from raw FFT to more sophisticated acoustic representations, such as **Mel-Frequency Cepstral Coefficients (MFCC)** or **Spectrograms**, which provide models with a richer time-frequency representation of the sound.

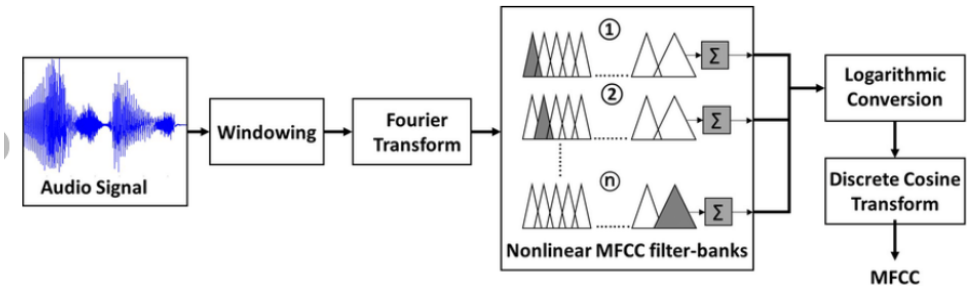


Figure 16: The MFCC feature extraction process

13 Context-Aware Sequence Decoding with Hidden Markov Models

Currently, our architecture classifies every keystroke independently. However, humans type in continuous words, not random characters. A **Hidden Markov Model (HMM)** integrated with a language model can interpret keystrokes as a continuous sequence: if a sound is ambiguous between 'T' and 'R', but the previous keystrokes were 'T' and 'H', the language model contextually deduces that 'E' is far more probable to form the word "THE".

13.1 Mathematical Formulation

In an HMM, the keys being pressed are the hidden states $S = \{S_1, S_2, \ldots, S_T\}$, and the acoustic features are the observations $O = \{x_1, x_2, \ldots, x_T\}$. The **Viterbi Algorithm** finds the most probable key sequence by maximizing:

$$\hat{S} = \arg\max_{S} \; P(S \mid O) \tag{6}$$

Applying Bayes' theorem decomposes this into an Acoustic Model and a Language Model:

$$\hat{S} = \arg\max_{S} \left[P(O \mid S) \cdot P(S) \right] \tag{7}$$

where $P(O \mid S)$ is the **emission probability**—the likelihood that a key produces the observed sound—and $P(S)$ is the **transition probability**—the linguistic probability of one letter following another (e.g., $P(\text{'U'} \mid \text{'Q'})$ is high, $P(\text{'Z'} \mid \text{'Q'})$ is negligible). The transition matrix $\mathbf{A}$ encodes these probabilities:

$$a_{ij} = P(S_t = j \mid S_{t-1} = i), \quad \sum_{j=1}^{N} a_{ij} = 1 \tag{8}$$

$$b_j(x_t) = P(x_t \mid S_t = j) = \sum_{k=1}^{K} w_{jk}\, \mathcal{N}(x_t \mid \mu_{jk}, \Sigma_{jk}) \tag{9}$$

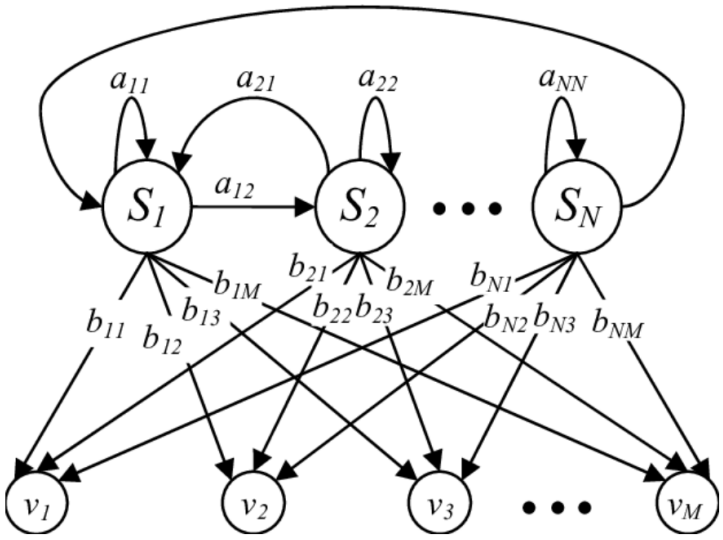


Figure 17: Example of a Hidden Markov Model

13.2 Proposed Hybrid Pipeline

Rather than discarding the existing SVM classifier, the most practical path is a **hybrid architecture** layering an HMM decoder on top of the current system:

1. **Retain** the existing segmentation and FFT feature extraction.
2. **Replace** the hard SVM output with a probabilistic output (top-k candidate keys with confidence scores).
3. **Construct** a transition matrix $\mathbf{A}$ from a large English text corpus.
4. **Apply** the Viterbi algorithm over the full keystroke sequence.

The pipeline transformation is summarized as:

$$\underbrace{\text{Keypress} \rightarrow \text{FFT} \rightarrow \text{Classifier} \rightarrow \text{Single Key}}_{\text{Current Pipeline (Independent)}} \implies \underbrace{\text{Keypress} \rightarrow \text{Features} \rightarrow \text{HMM} \rightarrow \text{Full Sequence}}_{\text{Proposed Pipeline (Context-Aware)}}$$

Even when individual keystroke classification confidence is low (60–70%), the Viterbi decoder can recover the correct sequence by leveraging linguistic constraints.

14 Related Academic Directions

Several parallel research threads present additional avenues for future exploration:

- **Context-Free Physical Attacks (2014):** Exploit keyboard geometry using Time Difference of Arrival (TDoA) between multiple microphones to infer key positions purely from physics ($\sim$72% accuracy).
- **VoIP-Based Remote Attacks (Skype & Type):** Keystroke sounds leak through Voice-over-IP calls, allowing passive extraction of typing data even over compressed audio channels.
- **Wavelet-Based Feature Extraction (2024):** Wavelet Transforms replace FFT for superior time-frequency localization, achieving $\sim$80% accuracy on keystroke classification.

Annex

All codes, batch automation scripts, and the trained Machine Learning AI models used to achieve full 104-key prediction have been made available on our GitHub repository.

Dataset Attribution:
Raw acoustic datasets utilized for evaluation originate from the Multi-Keyboard Acoustic (MKA) Dataset, provided under the **Creative Commons Attribution 4.0 International License (CC BY 4.0)**.

They can be consulted by scanning this QR code:

or by manually navigating to our repository structure at:
https://github.com/OussamaAKHAIL/EchoStrike

Figure Attributions (Chapter 8):

- Hidden Markov Model diagram sourced from ResearchGate:
  2008,Crop Type Recognition Based on Hidden Markov Models of Plant Phenology
- MFCC feature extraction process diagram sourced from ResearchGate:
  2021, Mel frequency cepstral coefficient temporal feature integration for classifying squeak and rattle noise